

Processamento paralelo de lista Encadeada com OpenMP Task

Para essa atividade, foi necessário implementar uma lista simplesmente encadeada em C. Segue a definição e “métodos” associados:

```
struct Node {
    char *file;
    struct Node *next;
} typedef Node;

Node *create_node(char *filename) {
    Node *new_node = (Node *)malloc(sizeof(Node));
    if (!new_node)
        return NULL;

    new_node->file = strdup(filename);
    new_node->next = NULL;
    return new_node;
}

Node *append_node(Node *head, char *filename) {
    Node *new_node = create_node(filename);
    if (head == NULL) {
        head = new_node;
        return head;
    }
    Node *temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = new_node;

    return temp;
}
```

O `create_node` faz exatamente o que se espera, enquanto o `append_node` além de adicionar um novo item na lista, também retorna um ponteiro para o item adicionado. Isso vai ser útil pra conseguir fazer a adição de novos nós na lista em $O(1)$ na prática.

Para a inicialização, aproveito o fato que o `append_node` retorna o último item para já dar um `append_node` diretamente nele na iteração seguinte:

```
Node *list = create_node("nó raiz");
Node *temp = list;

for (int i = 0; i < size; i++) {
    char buffer[MAX_FILENAME];
    snprintf(buffer, sizeof(buffer), "arquivo_%d.txt", i);
```

```
temp = append_node(temp, buffer);
}
```

Pesquisando um pouco sobre o `#pragma omp task`, ela é uma diretiva utilizada dentro de regiões paralelas, declarando que uma pool de threads devem ir pegando as tarefas que forem aparecendo conforme as threads vão ficando disponíveis. Uma implementação simples poderia ser:

```
// Define a região paralela
#pragma omp parallel
{
// Define a tarefa sendo todo o processo de percorrer a lista e processar cada nó.
#pragma omp task
{
    Node *current = list;
    while (current != NULL) {
        printf("Processando: %s | Thread ID: %d\n", current->file,
            omp_get_thread_num());
        current = current->next;
    }
}
}
```

O grande problema dessa abordagem é que eu vou processar todos os nós N vezes. O ideal é ter um “nó mestre” que vai iterar sobre o nome dos arquivos, e para cada arquivo eu criar uma task do openmp que processa aquele arquivo específico. Para isso, é utilizado o `#pragma omp single`, junto com o `#pragma omp task firstprivate(current)`, garantindo que cada task tenha seu arquivo para ser processado.

```
// define a região paralela
#pragma omp parallel
{
// Define que apenas uma thread deve fazer iterar sobre a lista de nós
#pragma omp single
{
    Node *current = list;
    while (current != NULL) {
        // Define que a primeira que chegar aqui e somente ela, deve executar
        // esse bloco de código
#pragma omp task firstprivate(current)
        {
            printf("Processando: %s | Thread ID: %d\n", current->file,
                omp_get_thread_num());
        }
        current = current->next;
    }
}
}
```

Dessa forma é possível processar todos os nós sem repetições por threads diferentes. No output dessa última execução foi possível observar que a carga(tasks) foi balanceada entre as threads.